Enterprise One Kernels

Table of contents

Kernel Overview 2
JDENET processes and kernel types 2
JDENET 2
The JDENET settings 3
General troubleshooting steps for Kernels: 4
Types of Kernel processes and their performance tuning recommendations 6
JDENET Reserved Kernel 6
UBE Kernel 6
Replication Kernel 8
Security Kernel 8
Lock Manager Kernel 10
Call Object Kernel 11
JDBNET Kernel 13
Package Install Kernel 14
Management Kernel 14
Scheduler Kernel 16
Package Build Kernel 17
UBE Subsystem Kernel 18
Workflow Kernel 18
Queue Kernel 19
XML Trans Kernel 20
XML List Kernel 21
Event Notification Kernel 22
Interoperability Event Observer Kernel 23
Kernel 21 24
XML Dispatch Kernel 24
XTS Kernel 26
XML Service Kernel 26
Metadata Kernel 27
XML Publisher / BI Publisher Kernel 28
Troubleshooting XML Kernels 29
Parameters that each Kernel will always have in INI section 30
Monitoring Kernel Processes 31
Kernel Recycling 32
Helpful Solutions and Guides 33
Tabular representation of features introduced/deprecated based on Application Releases and Tools Realeases 34

Kernel Overview

A Kernel is an essential background process that runs on enterprise one application/Logic server and performs a specific set of functions and responds to requests from client machines, servers and third party software. In order to meet different business needs with different functional requirements in Enterprise One, a number of different Kernel definition types have been defined to address separate set of functions and are defined as kernel def types.

JDENET processes and kernel types

JDENET

JDENET is the message-oriented middleware that provides communication between the clients and server as well as server to server via a socket-based JDENET_n middleware. The JDENET middleware, running within the CNC architecture, supports the configuration of business function components for execution in the heterogeneous distributed computing environment that the CNC architecture supports.

When E1 Services are started on enterprise server, a parent JDENET_n process is initiated. In Enterprise One, requests from a client or server to a server are handled through JDENET_n processes to communicate the request between different types of kernels. This parent process initiates other child processes which are called kernel (JDENET_k) processes. Any of these net processes can also start new kernel processes as new requests are received. These JDENET_k processes can be configured in the JDE.INI file. A minimum of JDNET_n should be 2. One is the parent process and another is the child process. It is best to have at least 2 child JDENET_n (JDENET_k) process plus one parent JDENET_n. A single JDENET_n process can handle a total of 1024 connections at max.

The jdenet process or the parent process that is started when you start JDE services has the capability to launch the required kernels as and when request messages come in from clients.

When a request is made from Enterprise One client or server, the message queues, which are interprocess communication (IPC) resources, are generated by calls to the Operating system from Enterprise one processes. The packets are routed to the JDENET_n job from a client or another server, and are placed in a message queue based on the type of process. This architecture also allows business functions and UBEs to execute remotely.

Based on the message identifier the jdenet will decide which kernel would handle this message and if this kernel is not available it will be started automatically. The only kernels that we usually set to auto start are the Scheduler kernel (this is one kernel that has to be set to auto start due to design complications) and call object kernel (these are set to auto start only for performance purposes and they are not mandatory).

JDE.INI Definition

```
[JDENET]
serviceNameListen=6014
serviceNameConnect=6014
maxNetProcesses=10
maxNetConnections=800
netShutdownInterval=15
maxKernelProcesses=100
maxKernelRanges=32
netTrace=0
enablePredefinedPorts=0
enterpriseServerTimeout=60
netBroadcastAddress=INADDR_BROADCAST
useKeepAlive=0
netCoreDump=0
kernelDelay=0
krnlCoreDump=0
statsUpdateInterval=0
netChildCheck=5
maxNumSocketMsgQueue=250
maxIPCQueueMsgs=100
internalQueueTimeOut=30
FilePacketBufferSize=32768
netTemporaryDir=/JDEdwards/JDE_HOME/targets/MNC400/temp
```

The JDENET settings

serviceNameListen= The port number or service name that is used by JD Edwards Enterprise One to receive and send communication packets to other servers/clients. The port number will default depending on the E1 application release.

serviceNameConnect= The port number or service name that is used by JD Edwards Enterprise One to receive and send communication packets to other servers/clients. The port number will default depending on the E1 application release.

maxNetProcesses= Defines the maximum number of jdenet_n processes that can be running. You can increase the value for a server that is expecting a large number of concurrent users.

If you need two or more processes, note that the first process is used exclusively to handle new connections and distribute them evenly among the other processes. In this case, you must add an extra process to act as a broker. Thus, the number of connections each jdenet_n process will handle is defined by the formula:

(maxNetConnections) / (maxNetProcesses -1)

If you increase the maxNetConnections setting, you should also increase the maxNetProcesses setting accordingly. In general, allow at least one jdenet_n process per 250 concurrent users. For AS400, the default setting is at least one jdenet_n process per 200 concurrent users.

maxKernelRanges= The maxKernelRanges should always be as big as the number of kernel definitons we have.

General troubleshooting steps for Kernels:

Step 1: Identify which kernel is the main culprit.

- Even though multiple kernel failures may be reported, a single one could be responsible for starting all of them. You should generally start with the problem that occurred first since it could have been the cause of all the downstream troubles.
- We should not attempt to correct other kernels since that may end up breaking them.
- Some kernels like the security and metadata are heavily linked with other kernels so any failures in these would affect other kernels severely.
- The COB(Call Object Kernel) Kernel is tightly linked with the Win32 and JAS Clients. Any errors in the CO Kernel jde log files could be the cause of errors seen on the client or in the client logs (especially when these errors refer to the enterprise server).
- Track the time of errors reported in the kernel logs and find which one was earliest and is that a minor or major failure.
- Check the jdenet_n logs, these show good information with respect to failed kernels.

Step 2: Identify the element that crashed this kernel.

- Once you have found the main kernel failure get the logs from that kernel and concentrate mainly on this.
- A call stack is a great source of information for troubleshooting kernels.
- If this kernel does not show a call stack then search in all the jdenet_n logs there is bound to be a call stack there. Although, this will only be true if the kernel that crashed was running a Business Function (BSFN) at the time it crashed. This will generally be true for CO Kernels and UBEs.
- If there is absolutely no call stack, then we would have to check for operating system level logs like crash dumps. In AS400, the job logs will act as the crash dumps.

- E1: OS: How To Collect Crash Dumps In Windows? (Doc ID 870502.1)
- E1: KER: How To Capture Call Stacks From Core Files On UNIX (Doc ID 833013.1)

- Always search top down in the kernel log and bottom up in a call stack.

- Find the first error or failure in the kernel log or call stack.

- For AS400, one should look for the joblogs.

Step 3: Fix it?

- A kernel may fail due to various reasons like,

  - Memory reference problem where the kernel stepped on a memory block it does not have access to or occupied by some other process.
  - The C programming language is used to write business functions, so pointers play a major role in making them work as well as crashing them.
  - IPC errors.
  - Business function logic. This would affect mainly the call object kernel and UBEs since these are generally the processes running them.
  - Operating system resource problem, this would cause drastic failures in a short period of time.
  - Failure of the primary NET process (jdesnet in windows and jdenet_n in all other OS). This is the process that will generally show up first in SAW and will have the following message in its jde.log (the key being "entry 0"):
    - process 3352 <jdenet_n> registered in entry 0

- An actual problem in the kernel can only be fixed by development but that is rarely the case as our kernels are quite stable as such.

- A kernel process does not terminate for no reason, something actually causes it to terminate. This could be because of a bug in logic processing causing a crash, loss of environment handles, loss of connectivity to a database, etc...

- There is no one solution or fix for kernel crashes since depending on the trigger it would change.

- In reality the one statement to keep in mind for all of our kernel issues is "Fix the cause fix the kernel".

Step 4: "No two kernels are alike"

- So now you have identified what kernel is failing and where approximately does it fail.
- Depending on the type of the kernel you have just identified the troubleshooting steps also varies.

Types of Kernel processes and their performance tuning recommendations

Each kernel type is designed to perform a specific service. Here is a listing of all kernels and their definitions in JDE.INI. For most of the kernel types some tuning recommendations have been suggested.

JDENET Reserved Kernel

Used for internal purposes and testing. Value for this kernel process in INI should not be changed.

JDE.INI Definition

[JDENET_KERNEL_DEF1]
krnlName=JDENET RESERVED KERNEL
dispatchDLLName= jdenet.dll
dispatchDLLFunction=_JDENET_DispatchMessage@28
maxNumberOfProcesses=1
numberOfAutoStartProcesses=0

The above settings are for Windows 2000 and NT. If you use a different platform, use the following settings for dispatchDLLName and dispatchDLLFunction:

Platform	dispatchDLLName	dispatchDLLFunction
AS400	JDENET	JDENET_DispatchMessage
HP9000B	libjdenet.sl	JDENET_DispatchMessage
SUN or RS6000	libjdenet.so	JDENET_DispatchMessage

UBE Kernel

UBE Kernel receives requests from Clients and schedules batch jobs in the job queue. A UBE kernel inserts a record in the job table. The value of the job queue name field of the inserted record indicates the job queue assigned to the new batch job.

Whenever a UBE kernel inserts a record in the job table, the batch job status field of the inserted record is always set to 'W' (wait status). After inserting, for OneWorld queues, UBE kernel sends a message to the Queue Kernel to schedule the job. For iSeries native queues, the UBE kernel does a SBMJOB to the native queue.

UBE kernels are also responsible for retrieving all non-BI Publisher UBE output to clients, whether those outputs are PDF, CSV, logs, or other kinds of output.

JDE.INI Definition

[JDENET_KERNEL_DEF2]
krnlName=UBE KERNEL
dispatchDLLName=jdekrnl.dll
dispatchDLLFunction=_JDEK_DispatchUBEMessage@28
maxNumberOfProcesses=3
numberOfAutoStartProcesses=0

The above settings are for Windows 2000 and NT. If you use a different platform, use the following settings for dispatchDLLName and dispatchDLLFunction:

Platform	dispatchDLLName	dispatchDLLFunction
AS400	JDEKRNL	JDEK_DispatchUBEMessage
HP9000B	libjdeknet.sl	JDEK_DispatchUBEMessage
SUN or RS6000	libjdeknet.so	JDEK_DispatchUBEMessage

Recommendation

One kernel for every 60 users.

Common Issues and Documents related to UBE Kernels

- Performance and Tuning: UBE Performance and Tuning (Doc ID 748333.1)
- EnterpriseOne UBE Performance Tips (Doc ID 825373.1)
- E1: UBE: New utilities for Batch Monitoring Tools release 8.98 (Doc ID 803246.1)
- E1: UBE: How to turn on logging for UBE submitted on server from fat client/web? (Doc ID 662087.1)
- E1: SCHED: How to cause the UBE scheduler server to not catchup after being down (Doc ID 657826.1)
- E1: SEC: Errors in Call Object and UBE Kernel Logs for User Profiles (Doc ID 815013.1)
- E1: OS: UBE Kernel zombie upon submission to AS400 (Doc ID 796014.1)
- E1: UBE: UBEs die after printque filled up (Doc ID 636233.1)
- E1: UBE: Unable to Submit UBEs to AS/400 Enterprise Server (Doc ID 646270.1)
- E1: UBE: What Is the Process ID value specified in Work With Submitted Jobs? (Doc ID 639086.1)
- E1: UBE: Unable to run UBES on AS400 Enterprise Server (Doc ID 664626.1)
- E1: UBE: Failed to validate auth token error when submitting UBES to server (Doc ID 638685.1)
- E1: UBE: Configuring Multiple Job Queues for Use by OneWorld (Doc ID 644511.1)

Replication Kernel

This kernel processes data replication requests. This is mainly a throwback to previous releases. This kernel has been deprecated starting 8.9 and onwards.

Security Kernel

Security Kernel processes security server requests. Additionally it also handles all user profile and security related table updates via messaging from the CO Kernel (or Win32 Client) when the appropriate BSFN's are called.

- Security kernel provides security tokens to all E1 server processes like jdenet_k, runube, runbatch, porttest and also E1 client processes like fat client, JAS client, interop connector clients, RTE. So upon initialization the security kernel is contacted by all kernels.

- The security kernel will be notified by appropriate Business functions whenever modifications are made to user profile/security record/security tables.

- Security kernel is not multi-threaded.

- If you do not Auto Start any security kernel processes, generally at least one security kernel will start when services are brought up (to validate security for some of the E1 Server processes). However if you do not auto start any security kernel on server1 and the following section in JDE.INI have set up security server running on a different enterprise server (server2), then it will not start the security kernel.
  Jde.ini of server1
  [SECURITY]
  SecurityServer=server2 (not server1)

JDE.INI Definition

[JDENET_KERNEL_DEF4]
krnlName=SECURITY KERNEL
dispatchDLLName=jdekrnl.dll
dispatchDLLFunction=_JDEK_DispatchSecurity@28
maxNumberOfProcesses=2
numberOfAutoStartProcesses=1

The above settings are for Windows 2000 and NT. If you use a different platform, use the following settings for dispatchDLLName and dispatchDLLFunction:

Platform	dispatchDLLName	dispatchDLLFunction
AS400	JDEKRNL	JDEK_DispatchSecurity
HP9000B	libjdeknet.sl	JDEK_DispatchSecurity
SUN or RS6000	libjdeknet.so	JDEK_DispatchSecurity

maxNumberOfProcesses
Represents the maximum number of kernel processes that can be run on the server for a particular kernel type. This setting can be modified to tune performance.

numberOfAutoStartProcesses

Shows the number of kernel processes that will automatically be started when the services are brought up. If this setting is 0, then no processes will be started automatically but will spawn based on requests to the particular kernel type. The value defined for the numberOfAutoStartProcesses must be less than or equal to the maxnumberOfProcesses for the kernel type.
This value can be modified to tune the performance. This would be done in the case where it was beneficial to start the process up front rather than waiting for the process to spawn on request.

Recommendations

- One security kernel for every 60 to 70 users. This is a base point, you would have to tune this number based on your requirements. Specifically the number of UBE process also depends on this kernel.

- It is recommended to automatically start a reasonable number of security kernels but depends on environment. In an environment where workload is high and users all log in at the same time, it may be reasonable to set the value for AutoStartProcess equal to the value for maxNumberOfProcesses.On the other hand, in an environment where workload is low and work does not all start at the same time, setting the value for AutoStartProcess equal to half of the value for maxNumberOfProcesses may yield better results.

Common Issues and Documents related to Security Kernels

- E1: SEC: How To Avoid LDAP Related Messages In Security Kernel Log When Not Configured? (Doc ID 827956.1)
- E1: SEC: Changing the JDE User Password (Doc ID 626139.1)

Lock Manager Kernel

Designed to process Transaction Manager and Lock Manager requests. Lock Manager is utilized to monitor time/date stamp changes to records.

JDE.INI Definition

[JDENET_KERNEL_DEF5]
krnlName=LOCK MANAGER KERNEL
dispatchDLLName= jdekrnl.dll
dispatchDLLFunction =_TM_DispatchTransactionManager@28
maxNumberOfProcesses=1
numberOfAutoStartProcesses=0

The above settings are for Windows 2000 and NT. If you use a different platform, use the following settings for dispatchDLLName and dispatchDLLFunction:

Platform	dispatchDLLName	dispatchDLLFunction
AS400	JDEKRNL	TM_DispatchTransactionManager
HP9000B	libtransmon.sl	TM_DispatchTransactionManager
SUN or RS6000	libtransmon.so	TM_DispatchTransactionManager

Call Object Kernel

Call Object kernel runs the business functions on the server based on the message received from clients through JDENET. When an application calls the BSFN for the first time, JDENET will determine the type of message received and will route the request over to the call object kernel based on the BSFN message type. If the User has already established the call object kernel process, the JDENET will route the BSFN to the same COBK process.
If a UBE triggers the BSFN, it will run the BSFN within the UBE process. The only time the BSFN will run on the different COBK process is when the BSFN is OCM mapped to a different server.

Currently only Call Object Kernel supports multi-thread processing. All other kernels are meant to handle data/cache one at a time.

JDE.INI Definition

[JDENET_KERNEL_DEF6]
krnlName=CALL OBJECT KERNEL
dispatchDLLName=XMLCallObj.dll (or jdeknet.dll)
dispatchDLLFunction=_XMLCallObjectDispatch@28
maxNumberOfProcesses=10
numberOfAutoStartProcesses=1
ThreadPoolSize=20
ThreadPoolSizeIncrement=5
singleThreadedMode=N

The above settings are for Windows 2000 and NT. If you use a different platform, use the following settings for dispatchDLLName and dispatchDLLFunction:

Platform	**dispatchDLLName**	**dispatchDLLFunction**
AS400	XMLCALLOBJ	XMLCallObjectDispatch
HP9000B	libxmlcallobj.sl	XMLCallObjectDispatch
SUN or RS6000	libxmlcallobj.so	XMLCallObjectDispatch

singleThreadMode
Indicates whether multithreading is enabled. "Y" means multithreading is turned off; "N" means that it is turned on. ThreadPoolSize indicates the maximum number of threads to be allowed (per call object kernel).

ThreadPoolSizeIncrement
Indicates the number of threads to be initiated at start-up and also indicates the number of threads added when a new function call request arrives and no idle threads exist.

Difference between these two definitions

[JDENET_KERNEL_DEF6] krnlName=CALL OBJECT KERNEL dispatchDLLName=libxmlcallobj.so dispatchDLLFunction=XMLCallObjectDispatch	[JDENET_KERNEL_DEF6] krnlName=CALL OBJECT KERNEL dispatchDLLName=libjdeknet.so dispatchDLLFunction=JDEK_DispatchCallObjectMessage

The difference between "libjdeknet.so" and "libxmlcallobj.so" is that the first one is used in a normal installation. When you do an install the first time this will be the default value that comes in. The second one is for interoperability purpose; this will give your call object kernel ability to use interop business functions. If you are not using interop then also you can have the XML call object enabled, it will not affect your setup in anyway it will just enhance your call object kernel. libxmlcallobj.so can handle both 'conventional' (non-xml embedded requests) as well as requests that leverages xml API calls. Customers who are currently using or planning to use Integration's that leverages XML APIs are encouraged to set the kernel def 6 to use the libxmlcallobj.so. In terms of performance, there is very minor impact.

Recommendations

In general the recommendation for the ratio of COB(CallObject) kernels to interactive users is 1 kernel for 6-10 interactive users. This number may vary based on how "heavy" the interactive use is.

If there aren't many Business Function(BSFN) calls or the BSFN's are short and quick, then you might be able to get more than 10 users per COB Kernel. If there are a LOT of BSFN calls or the BSFN's are long-running, you may need fewer than 6 users per COB kernel. So, for example if you have 650 concurrent users, first recommendation would be to try somewhere between 65 and 109 COB kernels. However, if your users are currently satisfied with their response time, then you might just extrapolate your current ratio (e.g. if your current ratio is 7 users per COB Kernel, then for 650 concurrent users, you might try 93 COB Kernels).

The following 3 settings are merely a good starting point for tuning. Every customer's environment and user case is different and they may achieve better performance by tweaking these settings.

1 kernel for 6-10 interactive users.
ThreadPoolSize = 30

ThreadPoolSizeIncrement = 10

Set for 10-15users/COB Kernel without Vertex
Set for 5-10 users/COB Kernel if running with Vertex

If on iSeries increase these numbers by 10
E.g. 20-25 user/COK without Vertex on iSeries

Common Issues and Documents related to Call Object Kernels

- E1: KER: Troubleshooting Call Object Kernel Zombie Issues and Gathering Information for Oracle Support (Doc ID 837800.1)
- E1: SAW: How to use web SAW to turn on jdedebug for a user's call object kernel (Doc ID 662290.1)

JDBNET Kernel

JDBNET processes database requests using a client and server. It can also be configured to process server-to-server requests-one server functions as a JDBNET client and the Other server functions as a JDBNET server.

JDBNET eliminates the need for database-specific network software. All database requests are transported to the JDBNET server, processed in a local database, and the results are transported back to the JDBNET client.

JDE.INI Definition:

[JDENET_KERNEL_DEF7]
krnlName=JDBNET KERNEL
dispatchDLLName=jdekrnl.dll
dispatchDLLFunction=_JDEK_DispatchJDBNETMessage@28
maxNumberOfProcesses=1
numberOfAutoStartProcesses=0

The above settings are for Windows 2000 and NT. If you use a different platform, use the following settings for dispatchDLLName and dispatchDLLFunction:

Platform	**dispatchDLLNam e**	**dispatchDLLFunction**
AS400	JDEKRNL	JDEK_DispatchJDBNETMessage
HP9000B	libjdeknet.sl	JDEK_DispatchJDBNETMessage

SUN or RS6000	libjdeknet.so	JDEK_DispatchJDBNETMessage

Recommendation:

One kernel for every 50 to 60 users. One need not have this kernel running if they are not using JDBNET drivers.

Package Install Kernel

As the name suggests, the package install kernel processes package installation requests to the server. Package install kernel is only valid for releases prior to 8.12 and as of 8.12 this kernel is no longer used. This kernel is utilized when packages are deployed to the enterprise server.

Management Kernel

The kernel process type used to instantiate the embedded management agent. The embedded management agent reports runtime information between the enterprise server and the Server Manager Management console. This kernel setting should not be set larger than one.

JDE.INI Definition:

```
[JDENET_KERNEL_DEF32]
 krnlName=MANAGEMENT KERNEL
 dispatchDLLName=jdemgmt.dll
 dispatchDLLFunction=_ManagementDispatch@28
 maxNumberOfProcesses=1
 numberOfAutoStartProcesses=1
```

The above settings are for Windows 2000 and NT. If you use a different platform, use the following settings for dispatchDLLName and dispatchDLLFunction:

Platform	dispatchDLLName	dispatchDLLFunction
AS400	JDEMGMT	ManagementDispatch
HP9000B	libjdemgmt.sl	ManagementDispatch
SUN or RS6000	libjdemgmt.so	ManagementDispatch

Recommendation

Server manager configuration under general setting in enterprise server configuration

[MANAGEMENT]
jdeHome=C:\JDE_AGENT (this should be the standalone agent path on the enterprise server). This is a windows example. Check your agent home location for this value.
instanceName=EnterpriseServer (this should be the enterprise server instance name in server manager console). This is the Enterprise server Managed instance component.

Trouble-Shooting steps for missing Enterprise server Runtime Metrics on Server Manager Console

- Verify if management kernel is running or not.
  Look for "MANAGEMENT KERNEL" in all jde.log in enterprise server.
  The easiest way to do this is by opening all the jde.log files and conducting a search for Management Kernel string.

- If management kernel is running does its log has any error message?
  If there are any error messages like failed to load JVM, missing classes folder, then they need to be fixed based on the error message.

- Enable logging for embedded agent by setting the following value in C:\JDE_AGENT\config\agent.properties, where
  C:\JDE_AGENT is the management agent install folder.
  log.embedded.instances=true
  log.level=ALL

- You will have to re-start the Enterprise server Managed instance in question for the Embedded agent log to be created under the Agent_home\logs folder.

- Check for Java heap dumps or core dump files on the server if the Management kernel is not running or if the Kernel crashed.

- You can set the following setting in the Enterprise server JDE.INI.

  [JDENET]

  HandleKrnlSignals=0

  The Call Object Kernel recyscling should be turned off when handleKrnlSignals is set to 0. Otherwise lot of zombie kernels will show up when recycling kicks in. See E1: KER: Kernel Recycling Process Results in Multiple Zombie Call Object Kernels (Doc ID 889046.1)

- When the management kernel crashes, this setting will create a core file under system/bin32 folder.

Common Issues and Documents related to Management Kernels

- E1:SVM: Missing Enterprise Server Runtime Metrics on Server Manager Console Troubleshooting Steps. (Doc ID 867232.1)
- E1: SVM: Runtime Metrics is Unavailable When Firewall is Enabled (Doc ID 876702.1)
- JD Edwards EnterpriseOne Tools 8.98 Server Manager Guide, Update (Doc ID 705509.1)
- E1: SVM: How to turn on logging for interactive apps in Server Manager (Doc ID 658887.1)

Scheduler Kernel

This kernel handles scheduler application requests. The Scheduler launches batch processes in a server/environment/user combination, based on the information in the Job Master table (F91300). After Scheduler is started, JDENET keeps it in a wait state by calling the Scheduler dispatch function every minute with an idle message. This idle message allows the Scheduler process to check whether it should launch a job or monitor the jobs that are running. In addition, JDENET also sends the Scheduler any message sent from the workstation (for example, messages that the new job schedules have been added).

JDE.INI Definition

[JDENET_KERNEL_DEF10]
krnlName=SCHEDULER KERNEL
dispatchDLLName= jdekrnl.dll
dispatchDLLFunction=_JDEK_DispatchScheduler@28
maxNumberOfProcesses=1
numberOfAutoStartProcesses=1

The above settings are for Windows 2000 and NT. If you use a different platform, use the following settings for dispatchDLLName and dispatchDLLFunction:

Platform	dispatchDLLName	dispatchDLLFunction
AS400	JDEKRNL	JDEK_DispatchSchedule r
HP9000B	libjdeschr.sl	JDEK_DispatchSchedule

		r
SUN or RS6000	libjdeschr.so	JDEK_DispatchSchedule r

Recommendation

This kernel process setting in INI file should remain at the shipped value of 1.

Common Issues and Documents related to Scheduler Kernels

- E1: SCHED: How Scheduler Server Works When Daylight Saving Is In Effect? (Doc ID 833253.1)
- E1: SCHED: Commonly Asked Questions About EnterpriseOne (JDE) Scheduler (Doc ID 849457.1)
- E1: SCHED: Setup Scheduler on multiple servers (Doc ID 659655.1)

Package Build Kernel

This kernel is responsible for processing package build requests.

JDE.INI Definitions:

[JDENET_KERNEL_DEF11]
krnlName=PACKAGE BUID KERNEL
dispatchDLLName= jdekrnl.dll
dispatchDLLFunction =_JDEK_DispatchPkgBuildMessage@28
maxNumberOfProcesses=1
numberOfAutoStartProcesses=1

The above settings are for Windows 2000 and NT. If you use a different platform, use the following settings for dispatchDLLName and dispatchDLLFunction:

Platform	dispatchDLLName	dispatchDLLFunction
AS400	JDEKRNL	JDEK_DispatchPkgBuildMessage
HP9000B	libjdeknet.sl	JDEK_DispatchPkgBuildMessage
SUN or RS6000	libjdeknet.so	JDEK_DispatchPkgBuildMessage

UBE Subsystem Kernel

This kernel controls the submission of UBE subsystem requests. The UBE Subsystem Kernel should be set to one process as the role/functionality of this kernel is to Add/update/delete records from table F986113. If this kernel is setup to more than one subsystem kernel it will cause problems when adding/updating/deleting data against F986113.

JDE.INI Definition

JDENET_KERNEL_DEF12]
krnlName=UBE SUBSYSTEM KERNEL
dispatchDLLName=jdekrnl.dll
dispatchDLLFunction=_JDEK_DispatchUBESBSMessage@28
maxNumberOfProcesses=1
numberOfAutoStartProcesses=0

The above settings are for Windows 2000 and NT. If you use a different platform, use the following settings for dispatchDLLName and dispatchDLLFunction:

Platform	dispatchDLLName	dispatchDLLFunction
AS400	JDEKRNL	JDEK_DispatchUBESBSMessage
HP9000B	libjdeknet.sl	JDEK_DispatchUBESBSMessage
SUN or RS6000	libjdeknet.so	JDEK_DispatchUBESBSMessage

Recommendation

ONLY one subsystem kernel can exist on an EnterpriseOne server instance therefore in JDE.INI - maxNumberOfProcesses parameter should always be One.

Workflow Kernel

This kernel is responsible for handling workflow requests

JDE.INI Definition

[JDENET_KERNEL_DEF13]
krnlName=WORK FLOW KERNEL
dispatchDLLName=workflow.dll
dispatchDLLFunction=_JDEK_DispatchWFServerProcess@28

maxNumberOfProcesses=1
numberOfAutoStartProcesses=0

The above settings are for Windows 2000 and NT. If you use a different platform, use the following settings for dispatchDLLName and dispatchDLLFunction:

Platform	dispatchDLLName	dispatchDLLFunction
AS400	WORKFLOW	JDEK_DispatchWFServerProcess
HP9000B	libworkflow.sl	JDEK_DispatchWFServerProcess
SUN or RS6000	libworkflow.so	JDEK_DispatchWFServerProcess

Recommendation

One kernel for every 40 users.

Queue Kernel

Queue kernel handles the job queues. Queue Kernel Launches jobs based on the Queue configuration in F986130, which is configured by the application – P986130 (Work with Job Queues). After launching jobs, queue kernel monitors the jobs that run on the system, catching the ones that crash and dumping the crashed child processes call stacks to the queue kernel log, and moves crashed jobs to the 'E' status, and their logs to the PrintQueue directory.

JDE.INI Definition

[JDENET_KERNEL_DEF14]
krnlName=QUEUE KERNEL
dispatchDLLName=jdekrnl.dll
dispatchDLLFunction=_DispatchQueueMessage@28
maxNumberOfProcesses=1
numberOfAutoStartProcesses=0

The above settings are for Windows 2000 and NT. If you use a different platform, use the following settings for dispatchDLLName and dispatchDLLFunction:

Platform	dispatchDLLName	dispatchDLLFunction

AS400	JDEKRNL	DispatchQueueMessage
HP9000B	libqueueknl.sl	DispatchQueueMessage
SUN or RS6000	libqueueknl.so	DispatchQueueMessage

Recommendation

[NETWORK QUEUE SETTINGS]
QKOnIdle=300 (in seconds, every 300 seconds poll F986130 (queue kernel config) and F986110(job master) to see if the table's data reflects the internal status properly. Never reduce to smaller than 15-20 seconds)

[DEBUG]
QKLog=0 (1 = some logging, 2=detailed analysis logging)

ONLY one queue kernel can exists on EnterpriseOne server instance therefore in JDE.INI - maxNumberOfProcesses parameter should always be One.

Common Issues and Documents related to Queue Kernels

E1: KER: General Recommendations for New Kernel Settings in JDE.INI (Doc ID 637939.1)

XML Trans Kernel

XML Trans kernel process handles XML requests. XML Transaction is XML-based interoperability that runs as an ERP kernel process. You can also use XML Transaction with a messaging adapter. XML Transaction interacts with interface tables (Z tables) to update the ERP database or to retrieve ERP data. You can create one XML document that includes both updates to ERP and retrieval of data from ERP.

JDE.INI Definition

Windows platform:
[JDENET_KERNEL_DEF15]
krnlName=XML TRANSACTION KERNEL
dispatchDLLName=XMLTransactions.dll
dispatchDLLFunction=_XMLTransactionDispatch@28
maxNumberOfProcesses=1
numberOfAutoStartProcesses=1

The above settings are for Windows 2000 and NT. If you use a different platform, use the following settings for dispatchDLLName and dispatchDLLFunction:

Platform	dispatchDLLName	dispatchDLLFunction
AS400	XMLTRANS	XMLTransactionDispatch
HP9000B	libxmltransactions.sl	XMLTransactionDispatch
SUN or RS6000	libxmltransactions.so	XMLTransactionDispatch

Common Issues and Documents related to XML Trans Kernels

For detailed explanation on this kernel process refer to the interoperability guide: JD Edwards EnterpriseOne Tools 8.98 Interoperability Reference Implementation Guide (Doc ID 705515.1)

XML List Kernel

XML List is XML-based interoperability that runs as an ERP kernel process. XML List provides List/GetNext functionality that allows you to collect a list of records from ERP. XML List built on the ERP table conversion (TC) engine.

XML List takes an XML document as a request and returns an XML document with the requested data. A list can represent data in a table, a business view, or data from a table conversion. Using data from a table conversion allows you to use multiple tables. By sending an XML document, you can retrieve metadata for a list, create a list, retrieve a chunk of data from a list, or delete a list. You can send the request through JDENet or third-party software to perform any of the following operations:

• Create List
• Get Template
• Get Group
• Delete List

XML List provides both trivial and non-trivial List/GetNext APIs. A trivial List/GetNext API performs simple gets such as selecting data from a single table. A non-trivial API uses additional functionality such as event rules. Each non-trivial List/GetNext BPAPI must have a table conversion designed for it. The data selection and data sequencing can be defined in an XML request at runtime.

XML List provides a list-retrieval engine that allows you to create an XML data file in the system repository and then retrieve the data in small chunks.

JDE.INI Definition

Windows Platform
[JDENET_KERNEL_DEF16]
krnlName=XML LIST
dispatchDLLName=xmllist.dll
dispatchDLLFunction=_XMLListDispatch@28
maxNumberOfProcesses=3
beginningMsgTypeRange=5257
endingMsgTypeRange=5512
newProcessThresholdRequest=0
numberOfAutoStartProcesses=1

The above settings are for Windows 2000 and NT. If you use a different platform, use the following settings for dispatchDLLName and dispatchDLLFunction:

Platform	dispatchDLLName	dispatchDLLFunction
AS400	XMLLIST	_XMLListDispatch
HP9000B	libxmllist.sl	_XMLListDispatch
SUN or RS6000	libxmllist..so	_XMLListDispatch

Common Issues and Documents related to XML LIST Kernels

For detailed explanation on this kernel process refer to the interoperability guide: JD Edwards EnterpriseOne Tools 8.98 Interoperability Reference Implementation Guide (Doc ID 705515.1)

Event Notification Kernel

Event Notification (EVN) kernel, also known as the event distributor is an ERP kernel process. The EVN kernel manages the subscribers and notifies them when an event occurs. The EVN kernel is shared by Z events, real-time events, and XAPI events. This kernel processes real-time events and XML documents generated by the Interoperability Event Observer, as well as Z file events. It publishes all E1 events to subscribers.

The enterprise server jde.ini file must be properly configured to support Z, real-time, and XAPI event generation. You use a text editor to manually edit and verify specific settings in the enterprise server jde.ini file.

To determine which kernels you need to set and for other jde.ini settings for each specific type of event, refer to the Configure the jde.ini File topics in the Events section of the Interoperability Guide.

JDE.INI Definition

[JDENET_KERNEL_DEF19]
krnlName=EVN KERNEL
dispatchDLLName=jdeie.dll
dispatchDLLFunction=_JDEK_DispatchITMessage@28
maxNumberOfProcesses=1
numberOfAutoStartProcesses=0

The above settings are for Windows 2000 and NT. If you use a different platform, use the following settings for dispatchDLLName and dispatchDLLFunction:

Platform	dispatchDLLName	dispatchDLLFunction
AS400	JDEIE	JDEK_DispatchITMessage
HP9000B	libjdeie.sl	JDEK_DispatchITMessage
SUN or RS6000	libjdeie..so	JDEK_DispatchITMessage

Common Issues and Documents related to Event Notification Kernels

For detailed explanation on this kernel process refer to the interoperability guide: JD Edwards EnterpriseOne Tools 8.98 Interoperability Reference Implementation Guide (Doc ID 705515.1)

Interoperability Event Observer Kernel

Processes information from business functions calling real-time APIs and uses that information to create an XML or a Z file that is publishable to subscribers by the Event Notification Kernel.

JDE.INI Definition

[JDENET_KERNEL_DEF20]
krnlName=IEO KERNEL
dispatchDLLName=jdeieo.dll
dispatchDLLFunction=_JDEK_DispatchIEOMessage@28
maxNumberOfProcesses=1
numberOfAutoStartProcesses=0

The above settings are for Windows 2000 and NT. If you use a different platform, use the following settings for dispatchDLLName and dispatchDLLFunction:

Platform	dispatchDLLName	dispatchDLLFunction
AS400	JDEIEO	JDEK_DispatchIEOMessage
HP9000B	libjdeieo.sl	JDEK_DispatchIEOMessage
SUN or RS6000	libjdeieo..so	JDEK_DispatchIEOMessage

Common Issues and Documents related to Interoperability Event Observer Kernels

For detailed explanation on this kernel process refer to the interoperability guide: JD Edwards EnterpriseOne Tools 8.98 Interoperability Reference Implementation Guide (Doc ID 705515.1)

Kernel 21

[JDENET_KERNEL_DEF21]
krnlName=OPE KERNEL
dispatchDLLName=jdeope.dll
dispatchDLLFunction=_BBOPE_DispatchMessage@28
maxNumberOfProcesses=1
numberOfAutoStartProcesses=0

XML Dispatch Kernel

Kernel for routing XML messages to their proper kernel. XML Dispatch is XML-based interoperability that runs as an ERP kernel process. The XML Dispatch kernel is the ERP central entry point for all XML documents.

For incoming XML documents, XML Dispatch identifies what kind of document is coming into ERP and sends the document to the appropriate kernel for processing. If XML Dispatch does not recognize the document, XML Dispatch sends the document to XTS to recognize and transform into native ERP format. After XTS transforms the document, the document is sent back to XML 41 Dispatch to be sent to the appropriate kernel for processing.

For outgoing documents, XML Dispatch is able to remember whether the request document was transformed into ERP native format. If the incoming request was transformed, then the outgoing response document is sent to XTS for transformation from native ERP format back into the format of the original request. After XTS transforms the document, the document is sent to XML Dispatch to distribute to the originator. The XML Dispatch kernel is able to route and load balance the XML documents.

For example, if you have many XML CallObject message types coming in at once, XML Dispatch will try to instantiate a new CallObject kernel. You set up the number of instances that a kernel can have in the jde.ini file. For example, if you set the number of instances for the CallObject kernel to 5, if more that one CallObject document comes into ERP, XML Dispatch sees that a particular kernel is busy and instantiates another one (up to 5). XML Dispatch is able to recognize new kernel definitions (such as XAPI) if the kernel is defined in the jde.ini file.

JDE.INI Definition

[JDENET_KERNEL_DEF22]
krnlName=XML Dispatch KERNEL
dispatchDLLName=xmldispatch.dll
dispatchDLLFunction=_XMLDispatch@28
maxNumberOfProcesses=1
numberOfAutoStartProcesses=0

The above settings are for Windows 2000 and NT. If you use a different platform, use the following settings for dispatchDLLName and dispatchDLLFunction:

Platform	**dispatchDLLName**	**dispatchDLLFunction**
AS400	XMLDSPATCH	JDEK_XMLDispatch
HP9000B	libxmldispatch.sl	JDEK_XMLDispatch
SUN or RS6000	libxmldispatch.so	JDEK_XMLDispatch

Common Issues and Documents related to XML Dispatch Kernels

For detailed explanation on this kernel process refer to the interoperability guide: JD Edwards EnterpriseOne Tools 8.98 Interoperability Reference Implementation Guide (Doc ID 705515.1)

XTS Kernel

Kernel for transforming XML messages from one type to another. The XTS Kernel must be defined in the server jde.ini file. The name of the configuration file is retrieved from the config_file system variable in the JVM, and these property settings are part of a configuration file other than jde.ini. The jde.ini file does not require any special configurations, other than defining the XTS Kernel.

JDE.INI Definition

[JDENET_KERNEL_DEF23]
krnlName=JDEXTS KERNEL
dispatchDLLName=xtskrnl.dll
dispatchDLLFunction=_JDEK_DispatchXTSMessage@28
maxNumberOfProcesses=1
numberOfAutoStartProcesses=0

The above settings are for Windows 2000 and NT. If you use a different platform, use the following settings for dispatchDLLName and dispatchDLLFunction:

Platform	dispatchDLLName	dispatchDLLFunction
AS400	XTSKRNL	JDEK_DispatchXTS
HP9000B	libxtskrnl.sl	JDEK_DispatchXTS
SUN or RS6000	libxtskrnl.so	JDEK_DispatchXTS

Common Issues and Documents related to XTS Kernels

For detailed explanation on this kernel process refer to the interoperability guide: JD Edwards EnterpriseOne Tools 8.98 Interoperability Reference Implementation Guide (Doc ID 705515.1)

XML Service Kernel

Processes inbound XAPI messages. The XML service kernel saves the XML document to disk, creates a unique handle, and then calls the callback business function that is provided in the DXAPIROUTE XAPI method ID element in the XML document. For specific information about the XML service kernel, see the online API documentation.

JDE.INI Definition

[JDENET_KERNEL_DEF24]
krnlName=XML Service KERNEL
dispatchDLLName=xmlservice.dll
dispatchDLLFunction=_XMLServiceDispatch@28
maxNumberOfProcesses=1
numberOfAutoStartProcesses=0

The above settings are for Windows 2000 and NT. If you use a different platform, use the following settings for dispatchDLLName and dispatchDLLFunction:

Platform	dispatchDLLName	dispatchDLLFunction
AS400	XMLSERVICE	JDEK_XMLServiceDispatch
HP9000B	libxmlservice.sl	JDEK_XMLServiceDispatch
SUN or RS6000	libxmlservice.so	JDEK_XMLServiceDispatch

Common Issues and Documents related to XML Service Kernels

For detailed explanation on this kernel process refer to the interoperability guide: JD Edwards EnterpriseOne Tools 8.98 Interoperability Reference Implementation Guide (Doc ID 705515.1)

Metadata Kernel

Processes XML spec access requests. The way that business functions process on the server has changed with release 8.12 and tools release 8.96. This is due to the change from TAM spec to XML spec files. The metadata kernel changes XML into C structures that will be readable by the call object kernel. The call object kernel is the only kernel that uses the metadata kernel. Other kernels contain an embedded JVM that performs the xml spec conversion.

JDE.INI Definition

[JDENET_KERNEL_DEF30]
krnlName=METADATA KERNEL

dispatchDLLName=MDSERIALIZ
dispatchDLLFunction=MetadataDispatch
maxNumberOfProcesses=1
numberOfAutoStartProcesses=1
ThreadPoolSize=50
ThreadPoolSizeIncrement=5

Recommendation

For large implementations it is a good practice to have one metadata kernel for every 300 users. This generally keeps the metadata kernels down to under four.

Common Issues and Documents related to Metadata Kernels
E1: KER: Metadata Kernel Performance and Tuning Guide (Doc ID - 820367.1)

XML Publisher / BI Publisher Kernel

This kernel handles XML publishing requests.

The following setting is the same regardless of which platform you are running
[JDE JVM]
XMLPVMOptions=-Djava.compiler=NONE -Xmx256M

JDE.INI Definition

The XML Publisher Kernel Definitions vary between platforms and are listed below:

Windows platform:

[JDENET_KERNEL_DEF31]
krnlName=XMLPUBLISHER KERNEL
dispatchDLLName=xmlpublisher.dll
dispatchDLLFunction=_XMLPDispatch@28
maxNumberOfProcesses=3
numberOfAutoStartProcesses=0

The above settings are for Windows 2000 and NT. If you use a different platform, use the following settings for dispatchDLLName and dispatchDLLFunction:

Platform	dispatchDLLName	dispatchDLLFunction
AS400	XMLPUBLISH	XMLPDispatch

HP9000B	libxmlpublisher.sl	XMLPDispatch
SUN or RS6000	libxmlpublisher.so	XMLPDispatch

SPECIFIC TO: EnterpriseOne 8.96, phase 1 and above

Common Issues and Documents related to XML Publisher/BI Publisher Kernels

- E1: XMLP: How to Obtain Jas Logs for the XML Publisher Kernel (Doc ID 782159.1)
- E1: XMLP: JDE.INI Kernel Settings for XMLP: Publisher (Doc ID 654237.1)
- E1: XMLP: Is There Specific Setup to Improve XML Publisher Performance? (Doc ID 761225.1)
- E1: XMLP: How to Encode Barcodes with BI Publisher for EnterpriseOne (Doc ID 782809.1)

Troubleshooting XML Kernels

If one or more XML kernels are not working properly, use the following troubleshooting guidelines to ensure that your system is correctly set up.

• Check the kernel definition in the server jde.ini file. Also check that the library name is correct for the platform on which you are running. Check the entry function name.

• Check that the kernel is allowed to start. Check the maxNumberOfProcesses and numberOfAutoStartProcesses values for the kernel in the server jde.ini file. It is not necessary to auto start kernels. To work with a particular kernel, the allowed number of processes should be one or more.

• If you have a large number of simultaneous requests made to a particular kernel type, increase the number of allowed processes for that kernel. This will not only reduce the turnaround time for requests but also eliminate any "Queue Full" errors.

• If you are using XMLList kernel, check that the LREngine section is correctly setup in the server jde.ini file, and that the specified path exists. Also check that the ERP user has write permission to this location.

• Check that the XML document is a well-formed XML document. To do this, use any XML editor or open the document in Microsoft Internet Explorer and check for errors.

• Check that the root of the input XML document is jdeRequest. All input XML documents should have jdeRequest as their root element.

• Check that valid user ID, password, and environment are provided in the XML document.

• Check that the request type in the XML document is correct. The allowed request types are callmethod, list, and trans for XMLCallObject, XMLList and XMLTransaction kernels, respectively.

Parameters that each Kernel will always have in INI section

The settings in these sections should not be changed except as noted:

KrnlName
Consists of a short description of the kernel's usage

dispatchDLLName
Identifies the name of the library that contains the function that will process requests received by the kernel process. These entries are platform specific.

dispatchDLLFunction
Contains the name of the function for handling messages. Technically this is the entry point into the jdekrnl.dll common library. You might see messages related to entry point if there is any problem with the jdekrnl.dll file. Each of the kernel types will have a different dispatchDLLFunction. These entries are also platform specific.

maxNumberOfProcesses
This represents the maximum number of kernel processes that can be run on this server for each kernel type. Some settings may need to be changed based on user count.

numberOfAutoStartProcesses
This represents the number of kernel processes that automatically start for each kernel type. If this number is 0, then no processes start automatically for that kernel type. This number must be less than the maximum number of processes for that kernel type. The user can modify this setting to tune performance. The default value is 0 for all JDENET_KERNEL_DEF sections.

Please see these guides for more information on the recommended INI settings as mentioned in solution
E1: KER: JDE.INI Tuning & Recommended JDE.INI Settings (From Development) (Doc ID 654975.1):

- Covers Client and Server JDE.INIs
- Solaris JDE.INI tuning Tool
- Covers iSeries Jde.INI Settings

Additional INI setting information can also be found in the Server and Workstation Administration guide which can be downloaded from Customer Connection website.

Monitoring Kernel Processes

We can take advantage of a number of tools available to us. Some of them are listed as follows:

- SAW
  SAW can be used from a fat client to monitor kernels on an enterprise server, view JDE and JDEDEBUG logs, find out which Process ID is assigned to each kernel and enable and disable debug logging on specific kernels in real-time without having to restart services. Some helpful SAW solution documents are listed below:

  - E1: SAW: How To Activate and Control Automated Emails from SAW (Doc ID 626210.1)
  - E1: SAW: How to enable jas debug dynamically for a single user with SAW 8.96 (Doc ID 652587.1)
  - E1: SAW: How to setup SAW admin rights (Doc ID 631783.1)

- Server Manager
  - Runtime Metrics display runtime information for Kernel Jobs. It displays the total number of Kernel processes (jdenet_k) that are concurrently running. Apart from kernel processes, it gives information for network jobs, kernel jobs (total number of network listener processes (jdenet_n), zombie processes, security kernel users, call object users.
  - Monitors in server manager are used to track events that occur within the management domain and record historical data of managed instance. More details about Monitors can be found in the Server Manager Guides that are referenced below.

- netwm tool - The netwm tool is delivered with the software and provides ability to dump information about the kernel processes.

- IPCDRVR_RL - As of 8.98, this is a diagnostic tool that has been delivered with the E1 System,. This tool allows an internal insight of the use of E1 IPC Resources.

These tools will retrieve specific information about the kernel processes, such as:

- The type of the kernel process: such as the Call object Kernels, which as type 6.

- Current state of the kernel process: is the process active or has become a zombie process.

- The process ID of the Kernel: This will allow us to get the following information:
  How much CPU the resource is taking as the process looping?
  How much memory the process is taking, do we potentially have a memory leak?

- The number of outstanding requests on the Kernel process. If we see a large number of outstanding requests, this can have several possible causes:

- There are not enough kernels to handle the load on the server. To verify this, look at the number of users on the individual kernel processes.
- The kernel process has started to loop. This can be caused by application specific issues and would be indicated by high CPU and low user counts. If it goes zombie or you kill the kernel it will collect information on the last application was using it. This is to isolate any application specific issues.
- The kernel Process has memory leak: This could be the result of an issue with a specific application or a tools related issue. To isolate this issue examine the memory consumption of the kernel process.

Kernel Recycling

These settings are used to configure predefined intervals in which the call object kernels will be restarted to prevent problems associated with very long running processes. The purpose of kernel recycling is to shut down old Call-Object (COB) kernels, and replace them with new kernels. The purpose is to minimize system-wide usage of resources. It does that by allowing the reclamation of cached memory, cached database connections, memory leaks, etc.

Kernel recycling provides a method for CallObject kernels to be automatically restarted under certain circumstances. This will allow for a more stable Enterprise Server by allowing these processes to clean up periodically and reduce the impact of any potential problems.

Intermittent kernel recycling provides fresh Call-Object kernels for long-running EnterpriseOne servers, without the need to re-start the entire EnterpriseOne system. The frequency of recycling can vary from once per week, down to minutes. Depending on the usage of the system, once per week or once per day should be adequate to minimize the impact of long-term, slow memory leaks and excessive caching. The best time of day to recycle would be in a lower usage period, following peak usage, mainly to reclaim cached resources. The least impact on users would also be during a low usage period.

Maximum cleanup of unused resources can be attained by using the settings for user inactivity and forced terminations. The user inactivity settings are designed to ignore users that are logged in, but not doing anything for long periods (inactivity time setting). The forced terminations will recycle the Call-Object kernels even if active users are logged in and active. Forced terminations help clean up disconnected user sessions that are still running on the server.

JDE.INI Definition

JDE.INI Kernel Recycling Parameters
[RECYCLING]
krnlRecycleTimeofDay= Sun: 03:30 23:00
krnlRecycleElapsedTime=
ianctiveUserTimeout= 6:00
timeToForcedExit= 12:00

krnlRecycleTimeofDay: Start to recycle kernel processes every Sunday at 3:30 AM and every day at 11:00 PM

Specifies the time of day to recycle the kernel. Multiple times may be specified although the maximum length of the INI entry is 150 characters. There are two valid formats for entries. Each entry is separated by a space character. The first format is DDD:HHH:MM. DDD represents the day of the week, which may be one of the following (without the quotes): 'Sun', 'Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', HH represents the hour of the day, in 24 hour notation. Valid values are 0 through 23. MM represents the minutes, which may be 0 through 59. The second format is HH:MM. The HH and MM follow the same criteria, but the day is omitted which specifies the recycle should be performed every day.

krnlRecycleElapsedTime: not set, using time-of-day setting above.

This entry defines the elapsed after startup in which a kernel should be required. It is specified in the format HHH:MM where HHH represents the number of hours and MM specifies the number of minutes. Valid values for the hours are 0 through 999, and valid values for the minutes are 0 through 59. For example the value '12:00' would specify the kernel should be recycled twelve hours after it is started.

ianctiveUserTimeout: user sessions considered inactive after 6 hours of inactivity.

The amount of time to consider a user as inactive. This value just be larger than the user session timeout value of any EnterpriseOne HTML servers that utilize the enterprise server. It is specified in the format HHH:MM where HHH represents the number of hours and MM specifies the number of minutes. Valid values for the hours are 0 through 999, and valid values for the minutes are 0 through 59.

timeToForcedExit: kernels forced to exit, if still running 12 hours after scheduled recycling time.

The amount of time to wait from a scheduled recycling until a forced exit. If not specified the kernel process will wait indefinitely if there are active users. It is specified in the format HHH:MM where HHH represents the number of hours and MM specifies the number of minutes. Valid values for the hours are 0 through 999, and valid values for the minutes are 0 through 59.

Helpful Solutions and Guides

- E1: KER: JDE.INI Tuning & Recommended JDE.INI Settings (From Development) (Doc ID 654975.1)
- E1: KER: Kernel Recycling is kicking active users out of their sessions
- E1: KER: Errors and Resolutions for AS400 PORTTEST (Doc ID 626235.1)
- E1: KER: How to monitor EnterpriseOne IPC resources (Doc ID 799714.1)

- E1: KER: How CheckKrnlHealth Setting In The JDE.INI Monitors Kernel Processes? (Doc ID 853693.1)
- E1: KER: Questions on Metadata, JDBNET and UBE Subsystem Kernels (Doc ID 779614.1)
- JD Edwards EnterpriseOne Tools Release 8.97 Reference Guides (Doc ID 702115.1)
- JD Edwards EnterpriseOne Tools 8.98 Reference Guides (Doc ID 705462.1)

Tabular representation of features introduced/deprecated based on Application Releases and Tools Realeases

Application Release	**Tools Release**	**Introduced**	**Deprecated**
	8.95	Multi threaded Call Object Kernel	
8.12	8.96	Metadata Kernel	Package Install Kernel
	8.97	▪ Server Manager (Management Kernel) ▪ Kernel Recycling	SAW Application
8.9			Replication Kernel